GCsync: Comprehensive Telemetry & Sensor Synchronization System

Ayush Mittal, Anwesha Adhikari, Kapil Karan Mathur, Aniket Pathak, Abhay Mall

1 Project Overview & Objectives

The primary objective of GCsync is to develop a high-performance, synchronized telemetry system capable of handling complex robotic sensor data in real-time. The system is designed to bridge the gap between raw hardware outputs and meaningful human-readable visualizations.

Key Objectives

- **Real-time Synchronization:** Ensure low-latency data flow between ROS (Robot Operating System) nodes and a centralized web dashboard.
- **Modular Architecture:** Implement a plugin-style architecture for sensors, allowing scalability from IMU tracking to LiDAR point clouds.
- **Persistent Analytics:** Provide a backend to record, store, and replay telemetry sessions for post-mission analysis.
- **Data Integrity:** Use TypeScript across the full stack to ensure strict type safety and reduce runtime errors in robotic monitoring tasks.

2 System Architecture

GCsync utilizes a decoupled architecture consisting of a Node.js/TypeScript backend and a React/Vite frontend.

2.1 Backend Implementation

The backend manages session persistence using MongoDB. It includes:

- **Models:** Schemas for Session and Telemetry data to track sensor logs.
- **Routes:** RESTful endpoints for managing live recording sessions and retrieving historical data.
- **Configuration:** Dynamic database connectivity managed via environment variables.

2.2 Frontend Implementation

The frontend is built for performance and modularity:

- **Core Logic:** Implements Factory Pattern for sensor instantiation and Strategy Pattern for handling LiDAR, IMU, Encoder data.
- **State Management:** Custom store hook manages global state, including ROS connections and telemetry streams.
- **Visualizations:** Specialized views for 3D rendering and real-time graphing.

3 Design & Diagrams

3.1 Class Diagram

The class diagram highlights the Strategy Pattern. The `BaseSensor` class is extended by specific implementations such as `LidarStrategy`, `ImuStrategy`, and `EncoderStrategy`, allowing unified data handling.

3.2 ER Diagram (Entity-Relationship)

The database schema focuses on the relationship between a **Session** and its associated **Telemetry** entries. Each telemetry point is linked to a session and categorized by sensor type.

3.3 Use-Case Diagram

The use-case diagram defines user interactions such as:

- Connecting to a ROS Master
- Starting/Stopping telemetry recording sessions
- Visualizing live LiDAR and IMU data
- Exporting session logs for analysis

4 The GCsync Dashboard

The Dashboard serves as the "Mission Control" for operators and integrates multiple panels into a unified interface.

Dashboard Features

- **Mission Control Panel:** Primary interface for session commands.
- **Telemetry Graphs:** Real-time line charts for sensor data.
- **System Graph:** Visual representation of active sensor nodes and status.

5 Technical Implementation Details

The project utilizes industry-standard design patterns:

- **Observer Pattern:** Used in EventBus to notify UI components of incoming sensor data without direct coupling.
- **Factory Pattern:** SensorFactory dynamically generates sensor handlers based on topic type.

6 Conclusion

GCsync provides a scalable, type-safe solution for robotic telemetry. By integrating real-time ROS communication with a modern web stack, the team has built a system capable of supporting advanced autonomous system development.